

Cloud Computing DevOps (CIS3032-N)

Marking Proforma

Student Details

Student Name:	Ali Mustafa	Student Id:	2110097
---------------	-------------	-------------	---------

Marking Criteria Signposting

Criterion	What to demonstrate	Where it can be found
Requirements implemented to be appropriately secure, distributed and resilient	Your containers are implemented with isolation techniques (e.g., abstract interfaces for external services)	• user-service/index.js lines 67–117 ◦ Auth0 Management API token acquisition and caching logic is isolated behind helper functions rather than being embedded directly in route handlers. • user-service/index.js lines 119–144 ◦ Account deletion logic (deleteAuth0User) encapsulates the external Auth0 Management API call. • product-service/index.js lines 780–813 ◦ External dependency (user-service) access is wrapped in try/catch with a safe fallback object. • user-service/index.test.js lines 172–219 ◦ External Auth0 Management API is replaced with mocked responses, demonstrating dev fakes/test doubles.
	Your containers participate in a distributed system (i.e., they make calls to other containers)	• product-service/index.js lines 760–813 ◦ GET /product-with-user performs an outbound HTTP request to the user-service using process.env.USER_SERVICE_URL. • docker-compose.yml ◦ Services are defined as separate containers with internal networking. • Environment configuration uses container service names rather than hardcoded addresses.
	Your containers continue to function when those they rely upon fail to respond (e.g., outbound HTTPS calls fail)	• product-service/index.js lines 780–813 ◦ If the user-service request fails, the catch block returns a fallback user object: ▪ { name: "Unavailable (Resilience Fallback)" } ◦ The endpoint continues responding successfully rather than crashing or timing out. • product-service/index.test.js lines 118–171 ◦ Unit test explicitly simulates user-service failure and asserts that the fallback behaviour occurs.

Criterion	What to demonstrate	Where it can be found
	Your containers are secured using appropriate authentication and authorization techniques	• user-service/index.js lines 11–35 ◦ Security middleware applied globally: ▪ helmet ▪ cors ▪ express-rate-limit • user-service/index.js lines 37–44 ◦ JWT validation middleware (checkJwt) configured using Auth0. • user-service/index.js lines 172–183 ◦ Protected route example enforcing authentication. • user-service/index.js lines 185–215 ◦ DELETE /me endpoint protected with JWT auth and authorization checks.

A comprehensive range of suitable industry-standard tools and frameworks used thoroughly and extensively	The range of software development tools, frameworks and libraries you have used when developing your containers	- Backend frameworks: - Express.js (user-service, product-service) - Frontend: - React (frontend application) - Authentication & security: - Auth0 (express-oauth2-jwt-bearer) - Helmet, CORS, rate limiting - Testing: - Jest, Supertest - Containerisation: - Docker, Docker Compose - CI/CD: - GitHub Actions - Security scanning: - Trivy (SARIF output) - Cloud hosting: - Azure (OIDC login and deployment) - Evidence available in: - package.json files - Dockerfiles .github/workflows/ci-cd.yml
	Thorough use of software development tools, frameworks and libraries you have used when developing your containers	.github/workflows/ci-cd.yml lines 1–171 - Automated testing, security scanning, and deployment. - index.test.js files across services - Demonstrates deep usage of Jest, mocks, and Supertest. - Movie captures: - CI pipeline execution (tests + Trivy scan) - Azure deployment - Running containers communicating with each other
Thorough and extensive automated verification testing demonstrated using test doubles to their full potential	Your containers are unit tested and the range of tests probe the range of possible input parameters and for failures within other components called	- user-service/index.test.js - Auth middleware behaviour /me deletion endpoint - Failure cases when Auth0 is not configured - product-service/index.test.js - Normal success paths - Dependency failure scenarios
	You have used suitable test doubles (e.g., fakes, mocks) in your unit tests	- user-service/index.test.js lines 3–24 - Auth middleware mocked. - user-service/index.test.js lines 172–219 - Auth0 Management API mocked for delete account behaviour. - product-service/index.test.js lines 118–171 - user-service HTTP calls mocked to simulate failure.

Suitable continuous software delivery pipeline through multiple environments with configuration demonstrated	Your containers' behaviours (e.g., which database to connect to, whether to use fakes, which auth server to use, etc.) are controlled via configuration which can be set without changing the code	• Use of environment variables throughout services: ◦ AUTHO_DOMAIN ◦ AUTHO_AUDIENCE ◦ USER_SERVICE_URL ◦ Azure deployment configuration • Evidence in: ◦ index.js files across services ◦ .env usage ◦ docker-compose.yml
	Build and release pipelines used to implement continuous development and/or continuous delivery of your containers	• .github/workflows/ci-cd.yml ◦ Lines 14–84: Automated tests ◦ Lines 85–105: Trivy security scan + SARIF upload ◦ Lines 106–171: Azure deploy using azure/login@v2 • Movie captures: ◦ Successful pipeline run ◦ Azure deployment completing ◦ Live services responding

Submission Checklist

Criterion	Checks	Notes
System Architecture (30%)	Have you completed and included your architecture diagram, including incorporating feedback from your tutor? Have you updated the diagram to show / comment upon how the final implementation differs from the intended? Have you included a copy of your agreed development plan? Have you included a README file?	Remember there should be one, final version of the architecture produced. All the artefacts produced for this criterion should be placed within the Documentation directory of your submission.
System Implementation (70%)	Have you completed and included the marking proforma (above section) for your software development work? Have you included a clone of each of your code repos (including any automated pipeline scripts / configuration)? Have you included movie captures demonstrating your software containers in use (via browser and/or Postman)? Do your movie captures demonstrate security and resilience? Have you included movie captured documenting a successful deployment of each of your software containers? Do your movie captures demonstrate all your DevOps tools? Have you included a short narrative (Word document) explaining and taking responsibility for the consequences of the technology related decisions?	The marking proforma should be submitted as part of the Documentation directory. The movie captures should be placed within the Media directory of your submission. The movie captures should effectively demonstrate each component communicating with the other components. The movie captures should demonstrate your DevOps practices and tools.